

MCQ-3

Topic:Exception Handling and Object Oriented Programming in Python

Student Name: _____ Date: _____ Score: _____

Instructions: Select the correct option by filling or marking the checkbox ☐ next to it.

1. 1. What will be the output of the following code?

```
class A: pass
class B(A): pass
class C(A): pass
class D(B,C):pass
print(D.__mor__)
```

- ☐ (D, B, C, A, object)
- ☐ (D, B, A, C, object)
- ☐ (D, C, B, A, object)
- ☐ (D, A, B, C, object)

2. Which statement is TRUE about Abstract Classes in Python?

- ☐ You cannot instantiate an abstract class even if it has no abstract methods
- ☐ A class becomes abstract only when it has at least one @abstractmethod
- ☐ Abstract classes cannot have concrete methods
- ☐ You can create objects if all abstract methods are implemented in the same

3. What is the output?

```
class A:
    def __init__(self): self.x = 10
class B(A):
    def __init__(self):
        super().__init__()
        self.x = 20
obj = B()
print(obj.x)
```

- ☐ 10
- ☐ 20
- ☐ Error
- ☐ None

4. Consider this code. What concept is primarily demonstrated?

```
class Engine: ...
class Wheel: ...
class Car:
    def __init__(self):
        self.engine = Engine()
        self.wheels = [Wheel() for _ in range(4)]
```

- ☐ Aggregation
- ☐ Composition
- ☐ Multiple Inheritance
- ☐ Polymorphism

5. In the context of MRO, what is the “diamond problem”?

- ☐ When two parent classes share a common grandparent, and MRO must resolve method lookup order unambiguously
- ☐ When a class inherits from more than two parents
- ☐ When abstract methods are not implemented
- ☐ When name mangling conflicts occur

6. What is the output of this code?

```
class A:
    def show(self): print("A")
class B(A):
    def show(self): print("B")
class C(A):
    def show(self): print("C")
class D(B, C):
    pass
d = D()
d.show()
```

- ☐ A
- ☐ B
- ☐ C
- ☐ B,C

7. Which of the following statements precisely describes the mechanical division of labor between `__new__` and `__init__` when an object is instantiated?

- ☐ `__init__` is an implicit static method that allocates memory addresses, whereas `__new__` is an instance method that modifies the newly allocated structural keys.
- ☐ `__new__` is the actual constructor method responsible for creating and returning a new instance, while `__init__` is an initializer method that takes that returned instance and configures its initial attributes.
- ☐ Python runs both methods in parallel on separate threads, allowing attribute initialization to execute concurrently alongside memory allocation.
- ☐ `__init__` executes first to validate incoming arguments, and only passes control to `__new__` if all parameter type-checks succeed.

8. In a multiple inheritance scenario, when `super()` is used to call a method, how does Python determine which method to call next?

- ☐ It searches the class hierarchy depth-first, evaluating from right to left
- ☐ It searches the class hierarchy depth-first, evaluating from left to right
- ☐ It traverses the immediate parents of the class where the call to `super()` originated, in the exact order
- ☐ It find the next class using method resolution order associated with the actual class.

9. Consider the following Python code snippet that implements a query builder using dunder methods. What will be output?

```
class Expression:
    def __init__(self, func):
        self.func = func

    def __call__(self, value):
        return self.func(value)

    def __and__(self, other):
        return Expression(lambda x: self(x) and other(x))

    def __invert__(self):
        return Expression(lambda x: not self(x))

class Query:
    def __init__(self, key):
        self.key = key

    def __eq__(self, value):
        return Expression(lambda d: d.get(self.key) == value)

    def __lt__(self, value):
        return Expression(lambda d: d.get(self.key) < value)

is_adult_staff = (Query("age") == 18) & ~(Query("role") == "Intern")

user_data = {"age": 18, "role": "Intern"}
result = is_adult_staff(user_data)
print(result)
```

- ☐ True
- ☐ False
- ☐ Type Error: "Expression" Object is not callable
- ☐ None

10. What is the exact output printed sequentially when `obj.test_calls()` is executed?

```
class Base:
    def __str__(self):
        return "Base Implicit"

    def render(self):
        return "Base Explicit"

class Sub(Base):
    def __str__(self):
        return "Sub Implicit"

    def render(self):
        return "Sub Explicit"

    def test_calls(self):
        print(str(super()))
        print(super().__str__())
        print(super().render())
```

```
obj = Sub()
obj.test_calls()
```

- ☐ Base Implicit
Base Implicit
Base Explicit
- ☐ <super: <class 'Sub'>, <Sub object>>
Base Implicit
Base Explicit
- ☐ <super: <class 'Sub'>, <Sub object>>
Sub Implicit
Base Explicit
- ☐ Sub Implicit
Sub Explicit
Base Implicit

11. What happens when you attempt to run the following Python code?

```
class Player:
    def __init__(self, name, score):
        self.name = name
        self._score = score

    @property
    def score(self):
        return self._score

p = Player("Alice", 150)
p.score = 200
print(p.score)
```

- ☐ It prints 200.
- ☐ It prints 150.
- ☐ It raises an `AttributeError: can't set attribute`.
- ☐ It raises a `SyntaxError: invalid decorator assignment`.

12. In software design, which statement precisely distinguishes the architectural implications of choosing Class Composition over Class Inheritance?

- ☐ Inheritance enforces a loose coupling interface contract (“has-a”), whereas Composition allows runtime modification of polymorphic bindings (“is-a”).
- ☐ Inheritance exposes the parent class’s internal details to subclasses, violating encapsulation (“white-box reuse”), while Composition preserves encapsulation by interacting strictly through public interfaces (“black-box reuse”).
- ☐ Composition maximizes compile-time code reuse by expanding static memory footprints, whereas Inheritance defers type checking to runtime interfaces.
- ☐ Composition enables strict structural type safety through structural subtyping, while Inheritance is required for nominal subtyping behaviors.

13. What is the exact sequential output printed to the console when the following Python script runs?

```
def calculate_score():
    try:
        print("Try Block")
        raise ValueError("Error Occurred")
    except ValueError:
        print("Except Block")
        return "Return Value"
    finally:
        print("Finally Block")

result = calculate_score()
print(result)
```

- ☐ Try Block
Except Block
Return Value
- ☐ Try Block
Except Block
Finally Block
Return Value
- ☐ Try Block
Finally Block
Except Block
Return Value
- ☐ Try Block
Except Block
Return Value
Finally Block

14. Which of the following statements precisely describes the theoretical behavior of Python's exception handling engine regarding the execution guarantee of the **finally** clause?

- ☐ If an unhandled exception occurs inside a **try** block, the **finally** block executes only if a matching **except** block catches and logs the error first.
- ☐ If a **finally** block executes a **return**, **break**, or **continue** statement, any exception that was actively raised and pending in the surrounding **try** block is discarded.
- ☐ The **finally** block runs concurrently on a separate OS thread to ensure resource cleanup can interrupt frozen loops inside the primary **try** block.
- ☐ If a **return** statement is encountered inside a **try** block, the function returns immediately, completely bypassing the code inside the **finally** block.

15. What is the exact outcome when the following Python script is executed?

```
from abc import ABC, abstractmethod

class Device(ABC):
    @abstractmethod
    def boot(self):
        pass

    def shutdown(self):
        return "Powering down"

class Laptop(Device):
    def run_diagnostics(self):
        return "Healthy"
```

```
try:
    obj = Laptop()
    print(obj.shutdown())
except TypeError as e:
    print("TypeError Raised")
```

- ☐ Powering down
- ☐ TypeError Raised
- ☐ AttributeError: 'Laptop' object has no attribute 'boot'
- ☐ Healthy

16. In Object-Oriented software engineering, which of the following statements precisely contrasts the design objectives of Abstraction and Encapsulation?

- ☐ Abstraction is focused on masking internal state modifications through compilation units, while Encapsulation is focused on dynamically overriding standard class inheritance interfaces.
- ☐ Abstraction hides internal complexity by exposing only essential, high-level features ("what" an object does), whereas Encapsulation bundles data and methods together while restricting direct access to the internal state ("how" an object protects its data).
- ☐ Abstraction requires the explicit deployment of an Abstract Base Class (ABC) in the code layout, while Encapsulation can only be achieved by utilizing name mangling rules.
- ☐ Abstraction restricts data visibility strictly within class constructors, whereas Encapsulation prevents methods from mutating local parameters.

17. What is the exact sequential output printed to the console when the following Python script is executed?

```
class SecureVault:
    def __init__(self, passcode):
        self.__passcode = passcode

    def view_vault(self):
        return f"Passcode: {self.__passcode}"

vault = SecureVault("Alpha99")

try:
    vault.__passcode = "Hacked123"
    print(vault.view_vault())
except AttributeError:
    print("Access Denied")
```

- ☐ Passcode: Hacked123
- ☐ Passcode: Alpha99
- ☐ Access Denied
- ☐ AttributeError: Private variable redefinition is forbidden

18. What is the exact sequential output printed to the console when the following Python script runs?

```
class Robot:
    def __init__(self, model):
        self.model = model
```

```

def fly(self):
    return f"{self.model} is airborne"

bot_1 = Robot("DroneX")
bot_2 = Robot("RoverY")

bot_1.action = fly

try:
    print(bot_1.action())
except TypeError:
    try:
        print(bot_1.action(bot_1))
    except Exception:
        print("Error Raised")

```

- ☐ DroneX is airborne
- ☐ Error Raised
- ☐ TypeError
- ☐ DroneX is airborne
DroneX is airborne

19. What will be the output of the following code?

```

class Animal:
    def sound(self):
        return "Some sound"

class Dog(Animal):
    def sound(self):
        return "Bark"

d = Dog()
print(d.sound())
print(super(Dog, d).sound())

```

- ☐ Some sound
Some sound
- ☐ Bark
Some sound
- ☐ Bark
Bark
- ☐ Error

20. What will be the output?

```

class Parent:
    def __init__(self):
        print("Parent Constructor")

class Child(Parent):
    def __init__(self):
        super().__init__()
        print("Child Constructor")

obj = Child()

```

- ☐ Child Constructor
- ☐ Parent Constructor
Child Constructor
- ☐ Parent Constructor
- ☐ Error: `super()` not used correctly